Paradigmas de Programação

Cálculo-λ: Lista 1

Parte A - Definição de cálculo-λ

1. Descreva a sequência de aplicações dos axiomas de construção que forma o termo-λ indicado. Assuma que x, y, z, w são variáveis e siga o modelo do exemplo, o qual ilustra a construção do termo $wz(\lambda y.xy)$:

$$x \xrightarrow{\text{aplicação}} xy \xrightarrow{\text{abstração}} \lambda y.xy \xrightarrow{\text{aplicação}} z(\lambda y.xy) \xrightarrow{\text{aplicação}} wz(\lambda y.xy)$$

(a) $(\lambda y.(\lambda x.xy)xy)$

(b) $\lambda w.xyx(\lambda z.z)$

Solução:

(a)

$$x \xrightarrow{\text{aplicação}} xy \xrightarrow{\text{abstração}} \lambda x.xy \xrightarrow{\text{aplicação}} (\lambda x.xy)x \xrightarrow{\text{aplicação}} (\lambda x.xy)xy \xrightarrow{\text{abstração}} \lambda y.(\lambda x.xy)xy$$

(b) O termo $\lambda w.xyx(\lambda z.z)$ requer a construção de dois subtermos, xyx e $\lambda z.z$, que são então combinados por aplicação e juntos por abstração em w:

$$z \xrightarrow{\text{abstração}} \lambda z.z$$

$$x \xrightarrow{\text{aplicação}} xy \xrightarrow{\text{aplicação}} xyx \xrightarrow{\text{aplicação}} xyx(\lambda z.z) \xrightarrow{\text{abstração}} \lambda w.xyx(\lambda z.z)$$

2. Para cada termo abaixo, indique se a substituição da variável x por N se aplica ou não. Em caso afirmativo, realize a substituição; caso contrário, justifique porque a substituição não se aplica.

(a) $(\lambda y.xyx)(yxy)$

(b) $\lambda x.(\lambda y.xyx)$

(c) $\lambda z.yzwz$

Solução:

(a) **Aplica-se.** A variável x ocorre livre em ambos os subtermos. Aplicando as regras de substituição para aplicação e abstração (supondo $y \notin FV(N)$ para evitar captura no contexto de λy):

$$\big((\lambda y.xyx)(yxy)\big)[x := N] = (\lambda y.xyx)[x := N] \cdot (yxy)[x := N] = (\lambda y.NyN)(yNy).$$

(b) **Não se aplica.** O símbolo λx mais externo ata a variável x em todo o corpo do termo; portanto, não há ocorrência *livre* de x sobre a qual a substituição possa agir.

(c) **Aplica-se, mas não altera o termo.** A variável x não ocorre no termo $\lambda z.yzwz$; pelas regras ii e iv (como no exemplo $(\lambda x.xy)[z := N] = \lambda x.xy$ dos slides):

$$(\lambda z.yzwz)[x := N] = \lambda z.\,(yzwz)[x := N] = \lambda z.yzwz.$$

3. Aplique os axiomas de redução, enquanto for possível, nas expressões abaixo:

(a) $(\lambda y.(\lambda x.xyx))x$

(b) $\lambda z.(\lambda y.yz)x$

Solução:

(a) Aplicamos a redução-β substituindo y por x no corpo $(\lambda x.xyx)$:

$$(\lambda y.(\lambda x.xyx))\, x \;\rightarrow_\beta\; (\lambda x.xyx)[y := x].$$

O termo substituto x tem $x \in FV(x)$ e y ocorre livre no corpo xyx de λx; logo a substituição direta faria λx capturar o x livre do argumento. Usamos a conversão-α para renomear a variável atada:

$$\lambda x.xyx \;\equiv_\alpha\; \lambda z.zyz.$$

Como $z \neq x$, a substituição não causa captura:

$$(\lambda z.zyz)[y := x] = \lambda z.\,(zyz)[y := x] = \lambda z.zxz.$$

O resultado $\lambda z.zxz$ é uma forma normal; nenhuma redução adicional é possível.

(b) Aplicamos a redução-β à subexpressão interna $(\lambda y.yz)x$:

$$\lambda z.\,(\lambda y.yz)\, x \;\rightarrow_\beta\; \lambda z.\,(yz)[y := x] = \lambda z.xz.$$

O termo $\lambda z.xz$ tem a forma $\lambda z.Mz$ com $M = x$ e $z \notin FV(x) = \{x\}$; a conversão-η é, portanto, aplicável:

$$\lambda z.xz \;\equiv_\eta\; x.$$

O resultado final é x.

4. Identifique, para cada termo abaixo, se a conversão-η é aplicável. Justifique caso não seja, ou aplique a conversão, se possível.

(a) $\lambda x.x$

(b) $\lambda x.y$

(c) $\lambda x.xyx$

Solução:

A conversão-η transforma $\lambda x.Mx$ em M. Ela só se aplica quando o corpo da abstração é uma *aplicação* da forma $M\,x$ com $x \notin FV(M)$.

(a) $\lambda x.x$: O corpo é a variável x, e não uma aplicação da forma $M\,x$. A conversão-η **não é aplicável**.

(b) $\lambda x.y$: O corpo é a variável y; tampouco é uma aplicação. A conversão-η **não é aplicável**.

(c) $\lambda x.xyx$: O corpo $xyx = (xy)x$ está na forma $M\,x$ com $M = xy$. No entanto, $x \in FV(xy) = \{x, y\}$, o que viola a condição $x \notin FV(M)$. A conversão-η **não é aplicável**.

5. Determine o conjunto das variáveis livres do termo $(\lambda x.zxyw)(\lambda y.wxzy)$.

Solução:

Pelas regras de variáveis livres FV:

$$FV(\lambda x.zxyw) = FV(zxyw) - \{x\} = \{z, x, y, w\} - \{x\} = \{z, y, w\},$$
$$FV(\lambda y.wxzy) = FV(wxzy) - \{y\} = \{w, x, z, y\} - \{y\} = \{w, x, z\}.$$

Logo:

$$FV\big((\lambda x.zxyw)(\lambda y.wxzy)\big) = \{z, y, w\} \cup \{w, x, z\} = \{x,\ y,\ z,\ w\}.$$

6. Escreva um combinador formado apenas por S e K que seja extensionalmente igual ao combinador K_*.

Solução:

O combinador $K_* \equiv \lambda xy.y$ seleciona sempre o *segundo* argumento, e SK se comporta da mesma forma:

$$SK\,x\,y \;=\; Ky\,(xy) \;=\; y \;=\; K_*\,x\,y,$$

pois $Sxyz = xz(yz)$ e $Kab = a$. Portanto, $K_* = SK$.

Parte B - Lógica booleana

7. Defina um termo lambda X que correponda à operação lógica "ou exclusivo", isto é, $Xpq = T$ se p e q tem valores lógicos distintos e $Xpq = F$, caso contrário.

Solução:

Usando $T \equiv \lambda xy.x$, $F \equiv \lambda xy.y$ e $\neg q \equiv qFT$, definimos:

$$X \;\equiv\; \lambda pq.\; p\,(\neg q)\,q \;=\; \lambda pq.\; p\,(qFT)\,q.$$

Verificação (tabela-verdade):

p	q	$X\,p\,q = p\,(qFT)\,q$	esperado
T	T	$T\underbrace{(TFT)}_{=F}T = TFT = F$	F ✓
T	F	$T\underbrace{(FFT)}_{=T}F = TTF = T$	T ✓
F	T	$F\underbrace{(TFT)}_{=F}T = FFT = T$	T ✓
F	F	$F\underbrace{(FFT)}_{=T}F = FTF = F$	F ✓

8. Escreva, para cada operador lógico abaixo, um combinador na base SK que seja extensionalmente igual ao operador. Indique a posição do operador (pré-fixada, in-fixada ou pós-fixada).

(a) condicional

(b) bicondicional

Solução:

Com $T = K$ e $F = SK$:

(a) **Condicional $p \to q$.** Combinador: K, posição pós-fixada.

$p\,q\,K = p \to q$:

- Se $p = T$: $T\,q\,K = q$ (T seleciona o primeiro argumento).
- Se $p = F$: $F\,q\,K = K = T$ (F seleciona o segundo argumento).

Verificação (tabela-verdade):

p	q	$p\,q\,K$	$p \to q$
T	T	$T\,T\,K = T$	T ✓
T	F	$T\,F\,K = F$	F ✓
F	T	$F\,T\,K = K = T$	T ✓
F	F	$F\,F\,K = K = T$	T ✓

Portanto, COND $= K$ (pós-fixado): $p\,q\,K \;=\; p \to q$.

(b) **Bicondicional $p \leftrightarrow q$.** Combinador: B, posição pré-fixada.

$p \leftrightarrow q$ é verdadeiro quando p e q têm o mesmo valor: para $p = T$ o resultado é q, e para $p = F$ o resultado é $\neg q$. O combinador pré-fixado

$$B \;\equiv\; \lambda p.\; p\, I\, \text{NOT},$$

onde $I = SKK$ e $\text{NOT} \equiv \lambda q.\; q\, F\, T = \lambda q.\; q\,(SK)\,K$ (a versão *pré-fixada* da negação; o $\mathbf{NOT} \equiv (\mathbf{SK})\mathbf{K}$ dos slides é *pós-fixado*, isto é, $q\,\mathbf{NOT} = q\,(SK)\,K$, e não serviria aqui, pois $(SK)K\,q = q$), satisfaz

$$B\,p\,q = (p\,I\,\text{NOT})\,q = \begin{cases} I\,q = q, & \text{se } p = T, \\ \text{NOT}\,q = \neg q, & \text{se } p = F. \end{cases}$$

Verificação:

p	q	$B\,p\,q$	$p \leftrightarrow q$
T	T	$I\,T = T$	T ✓
T	F	$I\,F = F$	F ✓
F	T	$\neg T = F$	F ✓
F	F	$\neg F = T$	T ✓

Derivação em SK pura:

$$B = \lambda p.\; p\, I\, \text{NOT} = S(\lambda p.\; p\, I)(\lambda p.\; \text{NOT}).$$

Como NOT não depende de p: $\lambda p.\,\text{NOT} = K\,\text{NOT}$. Traduzindo NOT para SK:

$$\text{NOT} = \lambda q.\; q\,(SK)\,K = S(\lambda q.\; q\,(SK))(\lambda q.\; K) = S\big(S\,I\,(K(SK))\big)(KK) = S\big(S(SKK)(K(SK))\big)(KK).$$

(Verificação: $S(SI(K(SK)))(KK)\,q = SI(K(SK))\,q\,(KKq) = Iq\,(K(SK)q)\,K = q\,(SK)\,K$.)
E, usando $I = SKK$:

$$\lambda p.\; p\, I = S(\lambda p.p)(\lambda p.I) = S\,I\,(K\,I) = S(SKK)(K(SKK)).$$

Portanto:

$$B = S\big(S(SKK)(K(SKK))\big)\Big(K\big(S(S(SKK)(K(SK)))(KK)\big)\Big).$$